
Timestamp Certificate

This timestamp was
created with *Ethereum*



originstamp

Timestamp

May-03-2025 06:00:47 UTC

Comment:

AQATVODTCC-3rd-May-2025.pdf



Hash:

611f0cbe02c90af36c97097972be147528a8d15fab65c2803fafa9a861ede5e9

Transaction:

[0x44c00105c915b89ac742dbdc7e3b70372d295282d15c1abf915091d347e74c89](https://etherscan.io/tx/0x44c00105c915b89ac742dbdc7e3b70372d295282d15c1abf915091d347e74c89)

Root Hash:

64ebe28b55c6871b0579e53287f40c94fffd156eef713687c312edae4dcde838

[Click here to verify your timestamp.](https://verify.originstamp.com)

This certificate is only valid in combination with the original file and OriginStamp's open procedure. More information on <https://verify.originstamp.com>.



Timestamp Certificate

Proof

The proof is necessary for the reproducibility of your document.

```
<?xml version="1.0" encoding="UTF-8"?>
<node type="key" value="64ebe28b55c6871b0579e53287f40c94fffd156eef713687c312edae4dcde838">
  <left type="mesh" value="a81b16e4fa8f8e70d44eb0460858fa48f9180a6ea13bed7f8d3dfd83bb1d8a6c">
    <left type="mesh" value="7dd9782a2c54b6b0906dad3ee4227f8b416b725fac9aa9ee98554f8fa4c6985">
      <left type="mesh" value="e56404290e70a11ee5787a4898aad587ba6630e377024596244f72fbcbe6e04"/>
      <right type="mesh" value="aef39799945ed799ad074afdfc554123e7136be671c00fd82b401dfaf85016c">
        <left type="mesh" value="3d83aae375d5c3ffc02f1950f48cc3501e8eba283cd49507a338a1754eaba945"/>
        <right type="mesh" value="dca7544fcdeb6a754f6eb232f121ec0d7135bcbbb8f64f140f9d665dac9e18c2">
          <left type="mesh" value="f5033a6c29680231f783e525f7b88ef965b35035c92db43200d102c604494698">
            <left type="mesh" value="2b685107bf3e313a3e0b67575d39a85e3effbe45e1ec13fe96d3553b7c79f257"/>
            <right type="mesh" value="50d563059c83041eac0b25fadd9f4f1725c7083bfcbb790d41060d33ad619c2b">
              <left type="mesh" value="4ba01b3988211d40f8f4fdbdc99d7e3318a51d77bee1aa77882b6af4c54cfa81"/>
              <right type="mesh" value="0e606bb15c4de982f78874a04bc683b42aa73e31f979d4942790a16ba96e3d5e">
                <left type="mesh" value="3e7ead2ea6728c8baabb89c0dfb3dd86466ec69ab6f136a700e51dea80badc22"/>
                <right type="mesh" value="71e52ca04dd7868b7b1d771b86a69eb69e19d30212e690d37651689eef57b195">
                  <left type="mesh" value="437aac4831fdfdbe5f4c8d85c18192945fc67d8e8fb2a8d7442af3b284e6d6af">
                    <left type="mesh" value="10bc57d530431819896feb0e54af38785241e17e1041e04a2c2c75141c934c5d">
                      <left type="mesh" value="1355b87263d2cbef108d681f8c2553616577fe1042e5f2fed2e64a8c439cb995">
                        <left type="mesh" value="351629b1b0f83c9e4997f9a4ad995f0dd1243f7b078b4e4c05d981b977f316b0">
                          <left type="mesh" value="9c9f65280361c1ef044346302d64e356c75f20dc0eee24542e211bdcfd1f10f0"/>
                          <right type="mesh" value="60882dd743539ac5fbb8e0cd7abd087ff6383d4d2ac473d830879b76fb42fb0e">
                            <left type="mesh" value="095d3202330c5d7f05de232342e4ccfb6216f55c4054b42bcd1d5ac780b9d49e"/>
                            <right type="mesh" value="f961735a477e911b47694ce964fcaa5b47b0415aa8981b9e935396f3032a4d84">
                              <left type="mesh" value="611e28d443d96176cc7bcf3c5e2c4f806ffe48d032169a8eccc118b6fcb5829"/>
                              <right type="hash" value="611f0cbe02c90af36c97097972be147528a8d15fab65c2803fafa9a861ede5e9"/>
                            </right>
                          </right>
                        </left>
                      </left>
                    </left>
                  </left>
                </left>
              </left>
            </left>
          </left>
        </left>
      </left>
    </left>
  </left>
  <right type="mesh" value="9d3da0024fde7775ebfdb9341633b684333cec097a60ee4c8575c63ead92decd"/>
  </right>
  <right type="mesh" value="421dc3f16d914c3e3d72f0b5676fad16b301af5847bb3fbface705a8ecd17e30"/>
  </right>
  <right type="mesh" value="45595a8b7e4c6ab9cd6e56822c4f9183de2584c97a7dfcfc20282b93b643940"/>
  </right>
  <right type="mesh" value="4b3e2826d624f1cb23d013d52260eb0e884ac7a547fc4da1a5f5440b95ce2125"/>
  </right>
  </right>
  </right>
  </left>
  <right type="mesh" value="7165cf49e486598baf861e578296eeae3ca03a10df4d41ecf552c71b497eaa6a"/>
  </right>
  </right>
  </left>
  <right type="mesh" value="5402df03068f7d971f9cd44495d92c003ce0d58c31aa1de9c663fb036d2484bb"/>
  </right>
  <right type="mesh" value="6a3b5a50c11ab1f75e65cb9c405d77af3e24cf097bed60186972c7f4fd359076"/>
</node>
```



Timestamp Certificate

Verification

OriginStamp is a timestamp service that uses various blockchains like the Bitcoin Blockchain to create tamper-proof timestamps for your data. Instead of backing up your data, OriginStamp embeds a cryptographic fingerprint in the blockchain. It is de facto impossible to deduce the content of your data from your fingerprint. Therefore, the data remains in your company and is not publicly accessible. All you need to do is send this fingerprint to OriginStamp via the interface. The integration of the RESTful API is very simple and convenient and allows all data to be easily tagged with a tamper-proof timestamp. This document shows the procedure and gives instructions for verifying a timestamp created with OriginStamp.

1. Determine the SHA-256 of your original file

There are numerous programs and libraries to calculate the SHA-256 of a file, such as [MD5FILE](#). Simply drag and drop or select your file, to retrieve the SHA-256 of your file.

2. Validate your proof

First, it must be verified that the hash of the original is part of the evidence. In order to check this, the proof can be opened with a conventional editor and its content can be searched for the hash. If the hash cannot be found, either the file was manipulated or the wrong evidence was selected.

3. Determine the root hash

The Merkle tree is a tree structure, that allows to organize the seed more efficient than a plain-text seed file. The tree is built from the bottom to the top and follows a defined schema. The value of a node is determined by the aggregated hash of its children.

Left child =

a8eb9f308b08397df77443697de4959c156fd4c68c489995163285dbd3eedaef

Right child =

ab95adaee8eb02219d556082a7f4fb70d19b8000097848112eb85b1d2fca8f67

Node = SHA-

256(a8eb9f308b08397df77443697de4959c156fd4c68c489995163285dbd3eedaefab95adaee8eb02219d556082a7f4fb70d19b8000097848112eb85b1d2fca8f67) =
47e47c96302eeba62ed443dd0c89b3411bbddd2c1ff6bdfb1f833fa1e060b85

This step is performed for all levels of the tree until the hash of the root has been calculated. If the hash of the root is identical as proof, the calculation was successful and the root hash is verified. The top hash corresponds to the root hash we embedded in the blockchain through a transaction. For a more detailed explanation of the Merkle tree, we want to refer to [Wikipedia](#).

4. Determine the Ethereum Transaction

Having determined the root hash in the previous step, we store this hash in a [Smart Contract in the Ethereum blockchain](#).

5. Check the transactions

The respective log events must be searched for each transaction in this contract. These events are divided into topics. The root hash should be contained in a topic. Caution: Some services display the hashes with a 0x prefix. As soon as the transaction has been found, the block time is the actual tamper-proof timestamp. To simplify the search, we added the transaction to the certificate.